

Step-by-Step Implementation Guide

Title - Network Log Analysis Using Hadoop

Technologies and Tools Used -

- HDFS
- MapReduce
- Hive
- Dataset: Kaggle — "Network Intrusion Detection" (KDD Cup 1999)
- Environment: Ubuntu (VirtualBox), Python 3, Hadoop 3.x

Step 1 – Choice of dataset

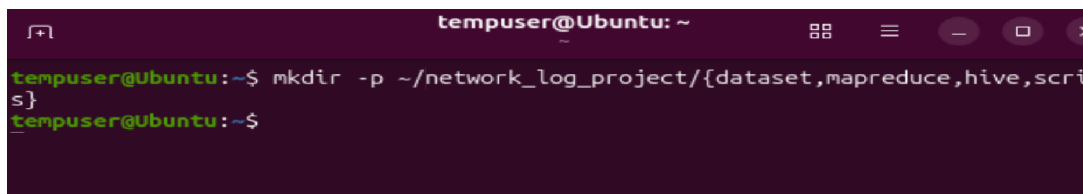
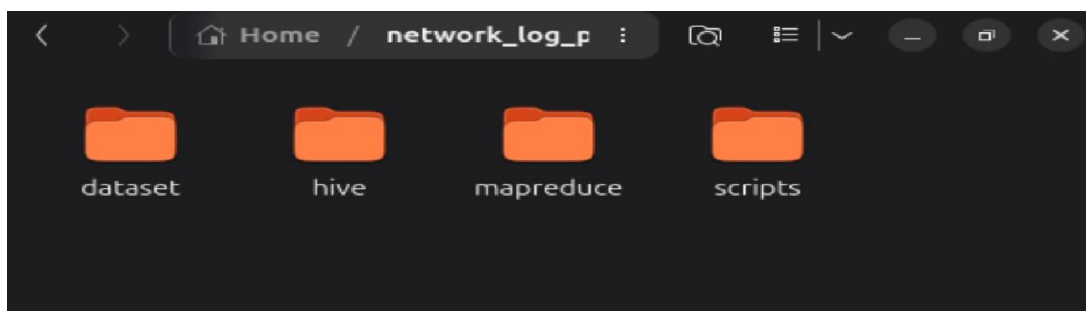
Dataset Name	KDD Cup 1999 Data — Network Intrusion Detection
Kaggle URL	https://www.kaggle.com/datasets/galaxyh/kdd-cup-1999-data
File to Use	kddcup.data_10_percent_corrected
Records	494,021 labeled network connection records
Format	CSV — no header row, 42 columns

Step 2 – Project Folder creation and sub folder creation

Script -

- `mkdir -p ~/network_log_project/{dataset,mapreduce,hive,scripts}`

Execution -

A terminal window with a dark purple background. The prompt is 'tempuser@Ubuntu: ~'. The command 'mkdir -p ~/network_log_project/{dataset,mapreduce,hive,scripts}' has been entered and executed. The prompt is now 'tempuser@Ubuntu: ~\$'.

Step 3 – Starting Hadoop and verifying it with JPS command

Script -

- `cd ~/network_log_project`
- `start-dfs.sh`
- `start-yarn.sh`
- `jps`

Execution -

```
tempuser@Ubuntu:~/network_log_project$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [Ubuntu]
tempuser@Ubuntu:~/network_log_project$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
tempuser@Ubuntu:~/network_log_project$ jps
9136 NodeManager
8465 NameNode
6657 MRAppMaster
9011 ResourceManager
8825 SecondaryNameNode
8606 DataNode
9503 Jps
```

Step 4 -

4.1 — Create the HDFS Directory and verifying it

Script -

- `hdfs dfs -mkdir -p /data/network/logs`
- `hdfs dfs -ls /data/network/`

Execution -

```
tempuser@Ubuntu:~/network_log_project$ hdfs dfs -mkdir -p /data/network/logs
tempuser@Ubuntu:~/network_log_project$ hdfs dfs -ls /data/network/
Found 1 items
drwxr-xr-x  - tempuser supergroup          0 2026-04-28 14:24 /data/network/logs
tempuser@Ubuntu:~/network_log_project$ s
```

4.2 — Upload the CSV File and verifying it.

Script -

- `hdfs dfs -put ~/network_log_project/dataset/network_logs.csv /data/network/logs/`
- `hdfs dfs -ls /data/network/logs/`
- `hdfs dfs -cat /data/network/logs/network_logs.csv | head -5`

Execution -

```
tempuser@Ubuntu: ~/network_log_project
tempuser@Ubuntu:~/network_log_project$ hdfs dfs -put ~/network_log_project/datas
et/network_logs.csv /data/network/logs
tempuser@Ubuntu:~/network_log_project$ hdfs dfs -ls /data/network/logs/
Found 1 items
-rw-r--r-- 1 tempuser supergroup 74889749 2026-04-28 14:27 /data/network/log
s/network_logs.csv
tempuser@Ubuntu:~/network_log_project$ hdfs dfs -cat /data/network/logs/network_
logs.csv | head -5 |
0,tcp,http,SF,181,5450,0,0,0,0,1,0,0,0,0,0,0,0,0,8,8,0.00,0.00,0.00,0.00,1
.00,0.00,0.00,9,9,1.00,0.00,0.11,0.00,0.00,0.00,0.00,0.00,normal.
0,tcp,http,SF,239,486,0,0,0,0,1,0,0,0,0,0,0,0,0,8,8,0.00,0.00,0.00,0.00,1.
00,0.00,0.00,19,19,1.00,0.00,0.05,0.00,0.00,0.00,0.00,0.00,normal.
0,tcp,http,SF,235,1337,0,0,0,0,1,0,0,0,0,0,0,0,0,8,8,0.00,0.00,0.00,0.00,1
.00,0.00,0.00,29,29,1.00,0.00,0.03,0.00,0.00,0.00,0.00,0.00,normal.
0,tcp,http,SF,219,1337,0,0,0,0,1,0,0,0,0,0,0,0,0,6,6,0.00,0.00,0.00,0.00,1
.00,0.00,0.00,39,39,1.00,0.00,0.03,0.00,0.00,0.00,0.00,0.00,normal.
0,tcp,http,SF,217,2032,0,0,0,0,1,0,0,0,0,0,0,0,0,6,6,0.00,0.00,0.00,0.00,1
.00,0.00,0.00,49,49,1.00,0.00,0.02,0.00,0.00,0.00,0.00,0.00,normal.
cat: Unable to write to output stream.
tempuser@Ubuntu:~/network_log_project$
```

Step 5 -

5.1 - Creating Python File to extract useful columns

Script -

- nano ~/network_log_project/scripts/parse_logs.py

Execution -

```
tempuser@Ubuntu: ~/network_log_project
tempuser@Ubuntu:~/network_log_project$ nano ~/network_log_project/scripts/parse_logs.py
tempuser@Ubuntu:~/network_log_project$
```

Python Code -

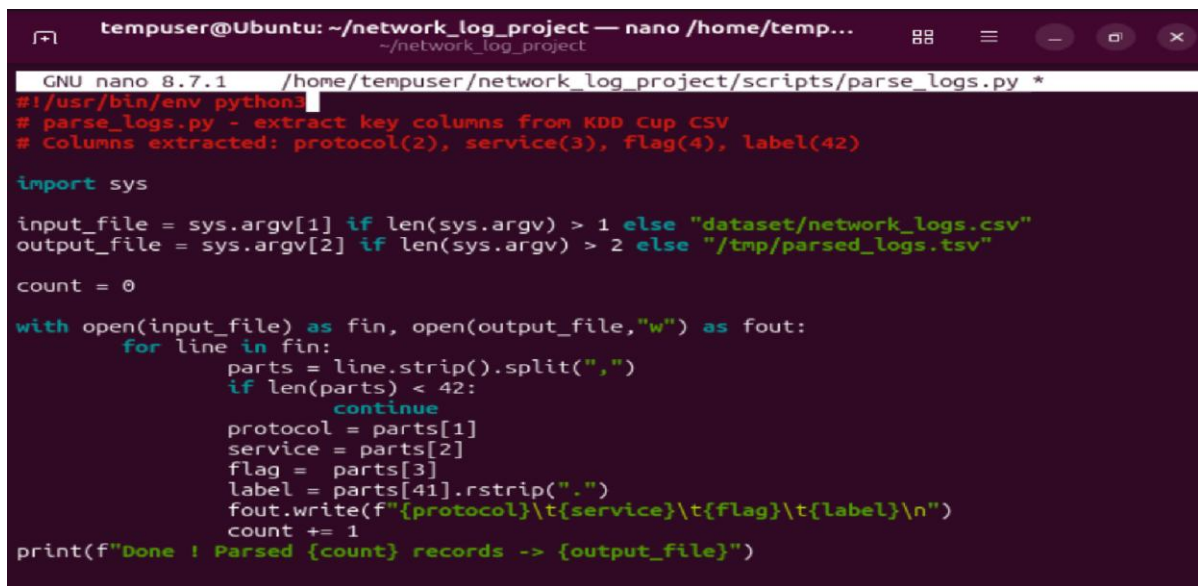
```
#!/usr/bin/env python3
# parse_logs.py — Extract key columns from KDD Cup CSV
# Columns extracted: protocol(2), service(3), flag(4), label(42)
import sys
input_file = sys.argv[1] if len(sys.argv) > 1 else "dataset/network_logs.csv"
output_file = sys.argv[2] if len(sys.argv) > 2 else "/tmp/parsed_logs.tsv"
count = 0
with open(input_file) as fin, open(output_file, "w") as fout:
    for line in fin:
        parts = line.strip().split(",")
        if len(parts) < 42:
```

```

        continue
    protocol = parts[1] # column 2
    service = parts[2] # column 3
    flag = parts[3] # column 4
    label = parts[41].rstrip(".") # column 42, remove trailing dot
    fout.write(f"{protocol}\t{service}\t{flag}\t{label}\n")
    count += 1
print(f"Done! Parsed {count} records → {output_file}")

```

Execution -



```

tempuser@Ubuntu: ~/network_log_project — nano /home/temp...
~/network_log_project
GNU nano 8.7.1 /home/tempuser/network_log_project/scripts/parse_logs.py *
#!/usr/bin/env python3
# parse_logs.py - extract key columns from KDD Cup CSV
# Columns extracted: protocol(2), service(3), flag(4), label(42)

import sys

input_file = sys.argv[1] if len(sys.argv) > 1 else "dataset/network_logs.csv"
output_file = sys.argv[2] if len(sys.argv) > 2 else "/tmp/parsed_logs.tsv"

count = 0

with open(input_file) as fin, open(output_file, "w") as fout:
    for line in fin:
        parts = line.strip().split(",")
        if len(parts) < 42:
            continue
        protocol = parts[1]
        service = parts[2]
        flag = parts[3]
        label = parts[41].rstrip(".")
        fout.write(f"{protocol}\t{service}\t{flag}\t{label}\n")
        count += 1
print(f"Done ! Parsed {count} records -> {output_file}")

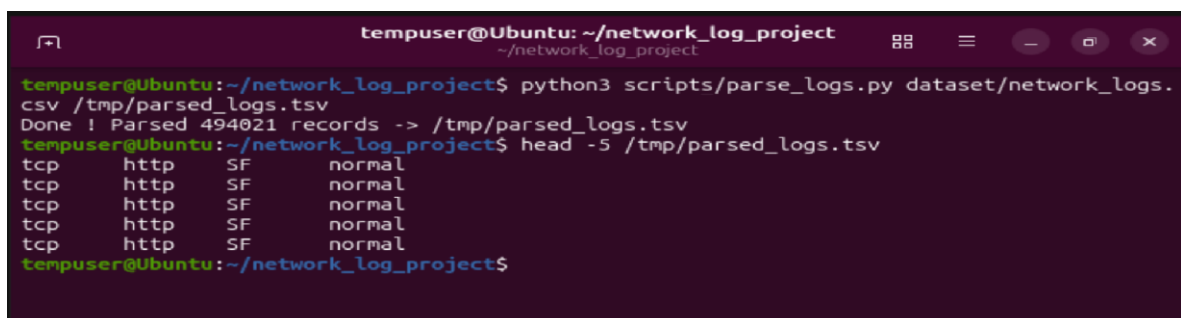
```

5.2 - Running the code and verify the output -

Script-

- python3 scripts/parse_logs.py dataset/network_logs.csv /tmp/parsed_logs.tsv
- head -5 /tmp/parsed_logs.tsv

Execution -



```

tempuser@Ubuntu: ~/network_log_project
~/network_log_project
tempuser@Ubuntu:~/network_log_project$ python3 scripts/parse_logs.py dataset/network_logs.csv /tmp/parsed_logs.tsv
Done ! Parsed 494021 records -> /tmp/parsed_logs.tsv
tempuser@Ubuntu:~/network_log_project$ head -5 /tmp/parsed_logs.tsv
tcp      http     SF       normal
tcp      http     SF       normal
tcp      http     SF       normal
tcp      http     SF       normal
tcp      http     SF       normal
tempuser@Ubuntu:~/network_log_project$

```

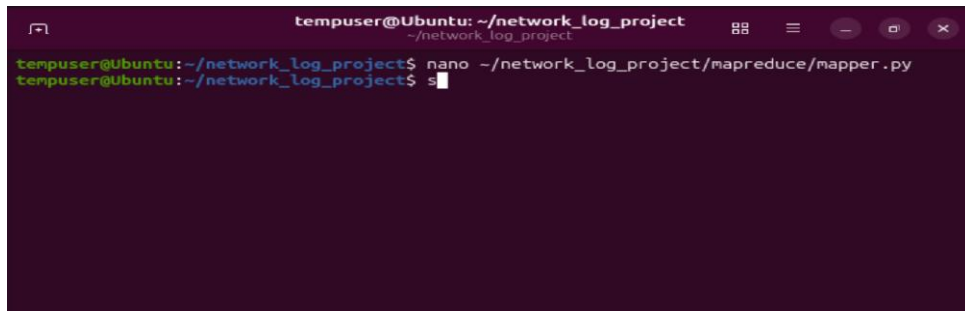
Step 6 -

6.1 - Mapper reads each log record and emits key-value pairs. Three types of keys are emitted per record — one for attack label count, one for protocol count, and one for service count.

Script -

- `nano ~/network_log_project/mapreduce/mapper.py`

Execution -

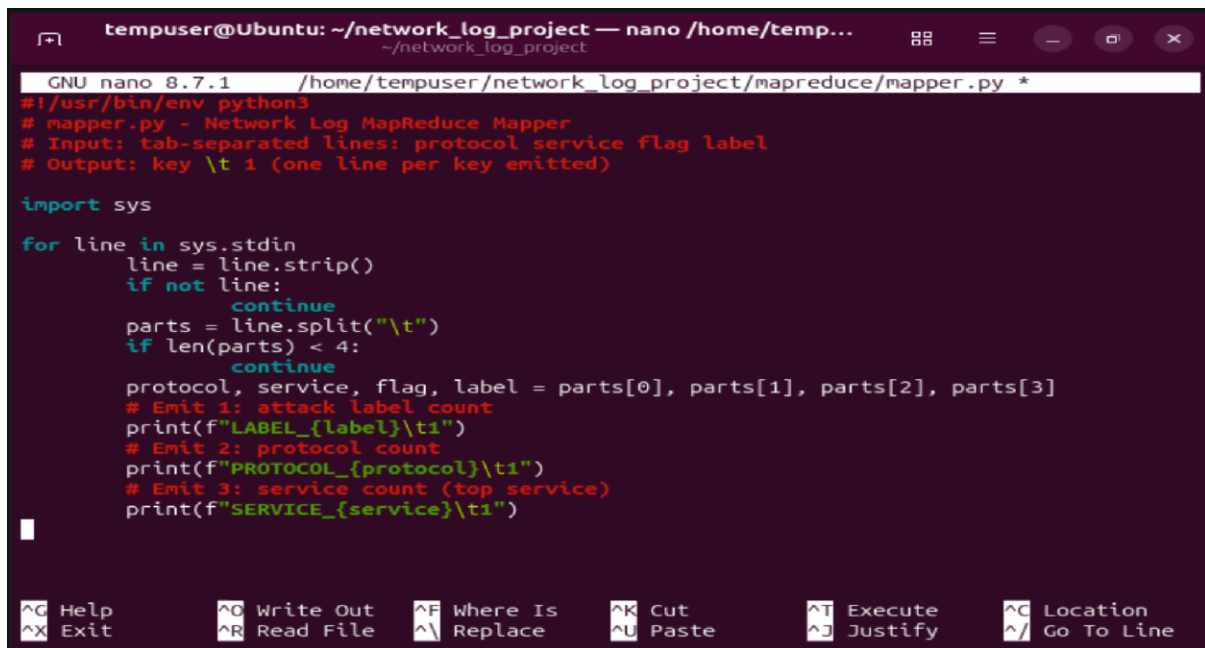
A terminal window with a dark purple background. The title bar shows 'tempuser@Ubuntu: ~/network_log_project'. The prompt is 'tempuser@Ubuntu:~/network_log_project\$'. The command 'nano ~/network_log_project/mapreduce/mapper.py' has been entered and executed, as indicated by the cursor on the next line.

```
tempuser@Ubuntu:~/network_log_project$ nano ~/network_log_project/mapreduce/mapper.py
tempuser@Ubuntu:~/network_log_project$ s
```

Python Code -

```
#!/usr/bin/env python3
# mapper.py — Network Log MapReduce Mapper
# Input: tab-separated lines: protocol service flag label
# Output: key \t 1 (one line per key emitted)
import sys
for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    parts = line.split("\t")
    if len(parts) < 4:
        continue
    protocol, service, flag, label = parts[0], parts[1], parts[2], parts[3]
    # Emit 1: attack label count
    print(f'LABEL_{label}\t1')
    # Emit 2: protocol count
    print(f'PROTOCOL_{protocol}\t1')
    # Emit 3: service count (top services)
    print(f'SERVICE_{service}\t1')
```

Execution -



```
tempuser@Ubuntu: ~/network_log_project — nano /home/temp...
~/network_log_project
GNU nano 8.7.1 /home/tempuser/network_log_project/mapreduce/mapper.py *
#!/usr/bin/env python3
# mapper.py - Network Log MapReduce Mapper
# Input: tab-separated lines: protocol service flag label
# Output: key \t 1 (one line per key emitted)

import sys

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    parts = line.split("\t")
    if len(parts) < 4:
        continue
    protocol, service, flag, label = parts[0], parts[1], parts[2], parts[3]
    # Emit 1: attack label count
    print(f"LABEL_{label}\t1")
    # Emit 2: protocol count
    print(f"PROTOCOL_{protocol}\t1")
    # Emit 3: service count (top service)
    print(f"SERVICE_{service}\t1")

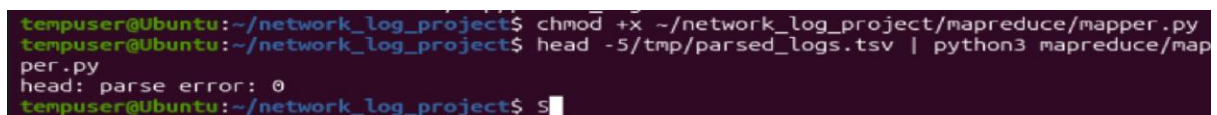
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
^_           ^C Location   ^/_ Go To Line
```

6.2 - Making mapper.py executable and running local before running on Hadoop

Script -

- `chmod +x ~/network_log_project/mapreduce/mapper.py`
- `head -5 /tmp/parsed_logs.tsv | python3 mapreduce/mapper.py`

Execution -



```
tempuser@Ubuntu:~/network_log_project$ chmod +x ~/network_log_project/mapreduce/mapper.py
tempuser@Ubuntu:~/network_log_project$ head -5 /tmp/parsed_logs.tsv | python3 mapreduce/map
per.py
head: parse error: 0
tempuser@Ubuntu:~/network_log_project$ S
```

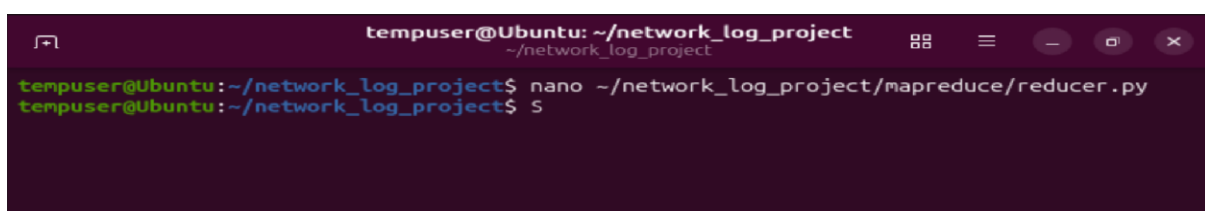
Step 7 -

7.1 - Reducer receives sorted key-value pairs from the Mapper and sums the counts for each unique key.

Script -

- `nano ~/network_log_project/mapreduce/reducer.py`

Execution -



```
tempuser@Ubuntu: ~/network_log_project
~/network_log_project
tempuser@Ubuntu:~/network_log_project$ nano ~/network_log_project/mapreduce/reducer.py
tempuser@Ubuntu:~/network_log_project$ S
```

Python Code -

```
#!/usr/bin/env python3
# reducer.py — Network Log MapReduce Reducer
# Input: sorted key \t count pairs (from mapper, sorted by Hadoop)
# Output: key \t total_count
import sys
current_key = None
current_count = 0
for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    try:
        key, count = line.split("\t", 1)
        count = int(count)
    except ValueError:
        continue
    if current_key == key:
        current_count += count        # same key: keep adding
    else:
        if current_key is not None:  # new key: emit the previous one
            print(f"{current_key}\t{current_count}")
            current_key = key
            current_count = count
        if current_key is not None:  # emit the last key
            print(f"{current_key}\t{current_count}")
```

Execution -

```
tempuser@Ubuntu: ~/network_log_project — nano /home/temp...  
~/network_log_project  
GNU nano 8.7.1 /home/tempuser/network_log_project/mapreduce/reducer.py *  
#!/usr/bin/env python3  
# reducer.py - Network Log Mapreducer Reducer  
# Input: sorted key \t count pairs (from mapper, sorted by Hadoop)  
# Output: Key \t total_count  
  
import sys  
  
current_key = None  
current_count = 0  
  
for line in sys.stdin:  
    line = line.strip()  
    if not line:  
        continue  
    try:  
        key, count = line.split("\t", 1)  
        count = int(count)  
    except ValueError:  
        continue  
  
    if current_key == key:  
        current_count += count  
    else:  
        if current_key is not None:
```

7.2 - Making reducer.py executable and running whole pipeline locally to test

Script -

- `chmod +x ~/network_log_project/mapreduce/reducer.py`
- `cat /tmp/parsed_logs.tsv | python3 mapreduce/mapper.py | sort | python3 mapreduce/reducer.py`

Execution -

```
tempuser@Ubuntu: ~/network_log_project  
tempuser@Ubuntu:~/network_log_project$ nano ~/network_log_project/mapreduce/reducer.py  
tempuser@Ubuntu:~/network_log_project$ chmod +x ~/network_log_project/mapreduce/reducer.py  
tempuser@Ubuntu:~/network_log_project$ cat /tmp/parsed_logs.tsv | python3 mapreduce/mapper  
.py | sort | python3 mapreduce/reducer.py  
LABEL_back 2203  
LABEL_buffer_overflow 30  
LABEL_ftp_write 8  
LABEL_guess_passwd 53  
LABEL_imap 12  
LABEL_ipsweep 1247  
LABEL_land 21  
LABEL_loadmodule 9  
LABEL_multihop 7  
LABEL_neptune 107201  
LABEL_nmap 231  
LABEL_normal 97278  
LABEL_perl 3  
LABEL_phf 4  
LABEL_pod 264  
LABEL_portsweep 1040  
LABEL_rootkit 10  
LABEL_satan 1589  
LABEL_smurf 280790  
LABEL_spy 2
```

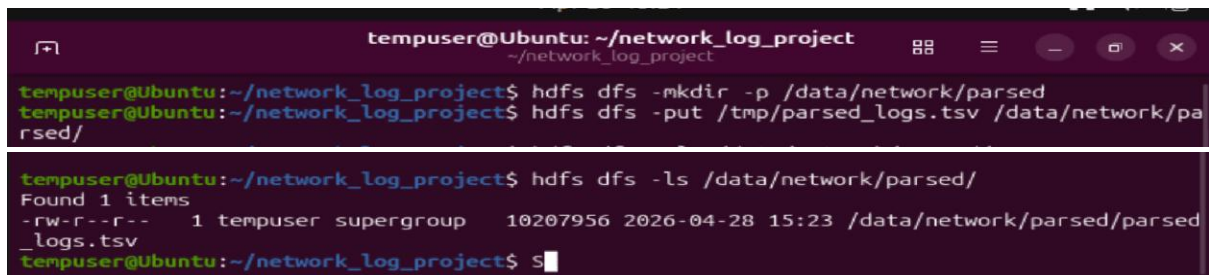

Step 8 – Running mapper and Reducer on Hadoop

8.1 - Upload Parsed Data to HDFS

Script -

- `hdfs dfs -mkdir -p /data/network/parsed`
- `hdfs dfs -put /tmp/parsed_logs.tsv /data/network/parsed/`
- `hdfs dfs -ls /data/network/parsed/`

Execution -



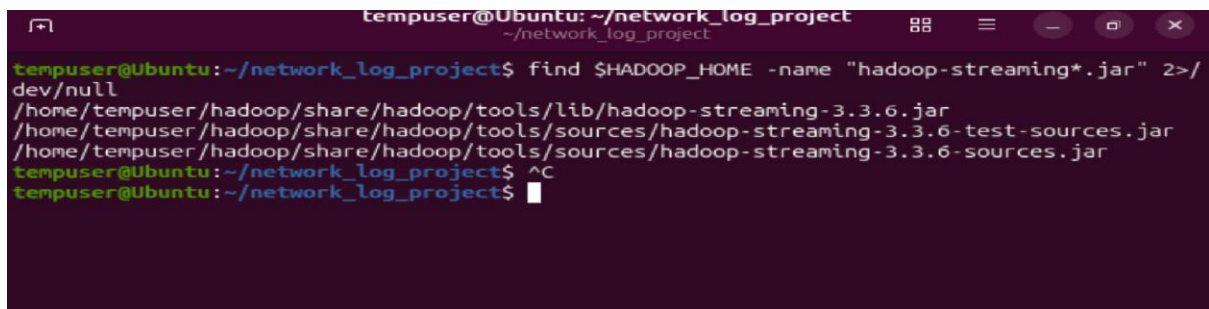
```
tempuser@Ubuntu: ~/network_log_project
tempuser@Ubuntu:~/network_log_project$ hdfs dfs -mkdir -p /data/network/parsed
tempuser@Ubuntu:~/network_log_project$ hdfs dfs -put /tmp/parsed_logs.tsv /data/network/parsed/
tempuser@Ubuntu:~/network_log_project$ hdfs dfs -ls /data/network/parsed/
Found 1 items
-rw-r--r-- 1 tempuser supergroup 10207956 2026-04-28 15:23 /data/network/parsed/parsed_logs.tsv
tempuser@Ubuntu:~/network_log_project$
```

8.2 — Find the Hadoop Streaming JAR

Script -

- `find $HADOOP_HOME -name "hadoop-streaming*.jar" 2>/dev/null`

Execution -



```
tempuser@Ubuntu: ~/network_log_project
tempuser@Ubuntu:~/network_log_project$ find $HADOOP_HOME -name "hadoop-streaming*.jar" 2>/dev/null
/home/tempuser/hadoop/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar
/home/tempuser/hadoop/share/hadoop/tools/sources/hadoop-streaming-3.3.6-test-sources.jar
/home/tempuser/hadoop/share/hadoop/tools/sources/hadoop-streaming-3.3.6-sources.jar
tempuser@Ubuntu:~/network_log_project$ ^C
tempuser@Ubuntu:~/network_log_project$
```

8.3 — Run the Hadoop Streaming Job

Script -

- `hadoop jar /home/tempuser/hadoop/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \`
- `-files /home/tempuser/network_log_project/mapreduce/mapper.py,/home`
- `/tempuser/network_log_project/mapreduce/reduce.py \`
- `-mapper "python3 mapper.py" \`
- `-reducer "python3 reducer.py" \`

- -input /data/network/parsed/parsed_logs.tsv \
- -output /data/network/mr_output

Execution -

```
tempuser@Ubuntu:~/network_log_project$ hadoop jar /home/tempuser/hadoop/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
-files /home/tempuser/network_log_project/mapreduce/mapper.py,/home/tempuser/network_log_project/mapreduce/reducer.py \
-mapper "python3 mapper.py" \
-reducer "python3 reducer.py" \
-input /data/network/parsed/parsed_logs.tsv \
-output /data/network/mr_output
packageJobJar: [/tmp/hadoop-unjar7267120098394215405/] [] /tmp/streamjob7209500773977410986.jar tmpDir=null
2026-04-28 17:54:27,470 INFO client.DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-04-28 17:54:27,790 INFO client.DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-04-28 17:54:28,074 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/tempuser/.staging/job_1777397589452_0005
2026-04-28 17:54:29,314 INFO mapred.FileInputFormat: Total input files to process : 1
2026-04-28 17:54:29,884 INFO mapreduce.JobSubmitter: number of splits:2
2026-04-28 17:54:30,065 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1777397589452_0005
2026-04-28 17:54:30,066 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-04-28 17:54:30,292 INFO conf.Configuration: resource-types.xml not found
2026-04-28 17:54:30,292 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-04-28 17:54:30,389 INFO impl.YarnClientImpl: Submitted application application_1777397589452_0005
2026-04-28 17:54:30,450 INFO mapreduce.Job: The url to track the job: http://Ubuntu:8088/proxy/application_1777397589452_0005/
2026-04-28 17:54:30,452 INFO mapreduce.Job: Running job: job_1777397589452_0005
2026-04-28 17:54:39,790 INFO mapreduce.Job: Job job_1777397589452_0005 running in uber mode : false
2026-04-28 17:54:39,800 INFO mapreduce.Job: map 0% reduce 0%
2026-04-28 17:55:03,673 INFO mapreduce.Job: map 100% reduce 0%
2026-04-28 17:55:11,787 INFO mapreduce.Job: map 100% reduce 100%
2026-04-28 17:55:12,814 INFO mapreduce.Job: Job job_1777397589452_0005 completed successfully
2026-04-28 17:55:12,931 INFO mapreduce.Job: Counters: 55
File System Counters
FILE: Number of bytes read=25986743
FILE: Number of bytes written=52814913
FILE: Number of read operations=0
FILE: Number of large read operations=0
FILE: Number of write operations=0
HDFS: Number of bytes read=10212270
```

8.4 — Check the MapReduce Output

Script -

- hdfs dfs -ls /data/network/mr_output/

Execution -

```
tempuser@Ubuntu:~/network_log_project$ hdfs dfs -ls /data/network/mr_output/
Found 2 items
-rw-r--r-- 1 tempuser supergroup 0 2026-04-28 17:55 /data/network/mr_output/_SUCCESS
-rw-r--r-- 1 tempuser supergroup 1673 2026-04-28 17:55 /data/network/mr_output/part-00000
tempuser@Ubuntu:~/network_log_project$
```

Script -

- hdfs dfs -cat /data/network/mr_output/part-r-00000

Execution -

```
tempuser@Ubuntu:~/network_log_project$ hdfs dfs -cat /data/network/mr_output/part-00000
LABEL_back 2203
LABEL_buffer_overflow 30
LABEL_ftp_write 8
LABEL_guess_passwd 53
LABEL_imap 12
LABEL_ipsweep 1247
LABEL_land 21
LABEL_loadmodule 9
LABEL_multihop 7
LABEL_neptune 107201
LABEL_nmap 231
LABEL_normal 97278
LABEL_perl 3
LABEL_phf 4
LABEL_pod 264
LABEL_portsweep 1040
LABEL_rootkit 10
LABEL_satan 1589
LABEL_smurf 280790
LABEL_spy 2
LABEL_tearndrop 979
LABEL_warezclient 1020
```

Script -

- `hdfs dfs -get /data/network/mr_output/part-r-00000 /tmp/mr_output.tsv`
- `cat /tmp/mr_output.tsv`

Execution -

```
tempuser@Ubuntu:~/network_log_project$ hdfs dfs -get /data/network/mr_output/part-00000 /tmp/mr_output.tsv
tempuser@Ubuntu:~/network_log_project$ cat /tmp/mr_output.tsv
LABEL_back 2203
LABEL_buffer_overflow 30
LABEL_ftp_write 8
LABEL_guess_passwd 53
LABEL_imap 12
LABEL_ipsweep 1247
LABEL_land 21
LABEL_loadmodule 9
LABEL_multihop 7
LABEL_neptune 107201
LABEL_nmap 231
LABEL_normal 97278
LABEL_perl 3
LABEL_phf 4
LABEL_pod 264
LABEL_portsweep 1040
LABEL_rootkit 10
LABEL_satan 1589
LABEL_smurf 280790
LABEL_spy 2
LABEL_tearndrop 979
LABEL_warezclient 1020
LABEL_warezmaster 20
PROTOCOL_icmp 283602
PROTOCOL_tcp 190065
PROTOCOL_udp 20354
SERVICE_IRC 43
SERVICE_X11 11
```

Step 9 - Analysis

9.1 - Start Hive

Script -

- `hive`

Execution -

```

tempuser@ubuntu:~/network_log_project$ hive
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/home/tempuser/hive/lib/log4j-slf4j-impl-2.17.1.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/home/tempuser/hadoop/share/hadoop/common/lib/slf4j-reload4j-1.7.36.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
Hive Session ID = 26a354e8-b658-422e-8365-ec0adac293a5

Logging initialized using configuration in jar:file:/home/tempuser/hive/lib/hive-common-3.1.3.jar!/hive-log4j2.properties Async: true
Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. spark, tez) or using Hive 1.X releases.
hive>

```

9.2 — Create Database and Table

Script -

- CREATE DATABASE IF NOT EXISTS network_logs_db;
- USE network_logs_db;
- DROP TABLE IF EXISTS log_events;
- CREATE EXTERNAL TABLE log_events (
 - protocol STRING,
 - service STRING,
 - flag STRING,
 - label STRING
)
 - ROW FORMAT DELIMITED
 - FIELDS TERMINATED BY '\t'
 - STORED AS TEXTFILE
 - LOCATION '/data/network/parsed';
 - SELECT * FROM log_events LIMIT 5;
 - SELECT COUNT(*) FROM log_events;

Execution -

```

hive> CREATE DATABASE IF NOT EXISTS network_logs_db;
OK
Time taken: 0.502 seconds
hive> USE network_logs_db;
OK
Time taken: 0.051 seconds
hive> DROP TABLE IF EXISTS logs_events;
OK
Time taken: 0.094 seconds

```

```

hive> CREATE EXTERNAL TABLE log_events (protocol STRING, service STRING, flag STRING, label STRING) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t' STORED AS TEXTFILE LOCATION '/data/network/parsed';
OK
Time taken: 0.222 seconds
hive> select * from log_events limit 5;
OK
tcp      http      SF        normal
tcp      http      SF        normal
tcp      http      SF        normal
tcp      http      SF        normal
tcp      http      SF        normal
Time taken: 2.041 seconds, Fetched: 5 row(s)
hive> select count(*) from log_events;
Query ID = tempuser_20260429155702_325eb9b1-2429-4034-b29a-b26ddaf1adf
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1777476933170_0001, Tracking URL = http://Ubuntu:8088/proxy/application_1777476933170_0001/
Kill Command = /home/tempuser/hadoop/bin/mapred job -kill job_1777476933170_0001
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-04-29 15:57:30,092 Stage-1 map = 0%, reduce = 0%
2026-04-29 15:57:47,616 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 7.88 sec
2026-04-29 15:57:55,159 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 10.25 sec
MapReduce Total cumulative CPU time: 10 seconds 250 msec
Ended Job = job_1777476933170_0001
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 10.25 sec HDFS Read: 10220457 HDFS Write: 106 SUCCESS
Total MapReduce CPU Time Spent: 10 seconds 250 msec
OK
494021
Time taken: 56.437 seconds, Fetched: 1 row(s)
hive>

```

9.3 — Analysis Query 1: Attack Type Distribution

Script -

- SELECT label,
COUNT(*) AS total_events,
ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM log_events), 2) AS pct
- FROM log_events
- GROUP BY label
- ORDER BY total_events DESC
- LIMIT 10;

Execution -

```

hive> SELECT label,
> COUNT(*) AS total_events,
> ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM log_events), 2) AS pct
> FROM log_events
> GROUP BY label
> ORDER BY total_events DESC
> LIMIT 10;

```

```

MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 5.24 sec HDFS Read: 10220235 HDFS Write: 716 SUCCESS
Stage-Stage-4: Map: 1 Reduce: 1 Cumulative CPU: 4.02 sec HDFS Read: 10219717 HDFS Write: 117 SUCCESS
Stage-Stage-5: Map: 1 Cumulative CPU: 1.96 sec HDFS Read: 8188 HDFS Write: 788 SUCCESS
Stage-Stage-3: Map: 1 Reduce: 1 Cumulative CPU: 3.62 sec HDFS Read: 8998 HDFS Write: 388 SUCCESS
Total MapReduce CPU Time Spent: 14 seconds 840 msec
OK
smurf 280790 56.84
neptune 107201 21.70
normal 97278 19.69
back 2203 0.45
satan 1589 0.32
ipsweep 1247 0.25
portsweep 1040 0.21
warezclient 1020 0.21
teardrop 979 0.20
pod 264 0.05
Time taken: 122.285 seconds, Fetched: 10 row(s)

```

9.4 — Analysis Query 2: Protocol Usage

Script -

- SELECT protocol,
COUNT(*) AS connections,
SUM(CASE WHEN label != 'normal' THEN 1 ELSE 0 END) AS attacks
- FROM log_events
- GROUP BY protocol
- ORDER BY connections DESC;

Execution -

```
Time taken: 47.173 seconds, Fetched: 3 row(s)
hive> SELECT protocol,
> COUNT(*) AS connections,
> SUM(CASE WHEN label != 'normal' THEN 1 ELSE 0 END) AS attacks
> FROM log_events
> GROUP BY protocol
> ORDER BY connections DESC;

MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 3.33 sec HDFS Read: 10222591 HDFS Write: 182 SUCCESS
Stage-Stage-2: Map: 1 Reduce: 1 Cumulative CPU: 2.36 sec HDFS Read: 8107 HDFS Write: 175 SUCCESS
Total MapReduce CPU Time Spent: 5 seconds 690 msec
OK
icmp 283602 282314
tcp 190065 113252
udp 20354 1177
Time taken: 47.173 seconds, Fetched: 3 row(s)
hive>
```

9.5 — Analysis Query 3: Top Services Targeted by Attacks

Script -

- SELECT service,
COUNT(*) AS total,
SUM(CASE WHEN label != 'normal' THEN 1 ELSE 0 END) AS attack_count
- FROM log_events
- WHERE label != 'normal'
- GROUP BY service
- ORDER BY attack_count DESC
- LIMIT 10;

Execution -

```
hive> SELECT service,
> COUNT(*) AS total,
> SUM(CASE WHEN label != 'normal' THEN 1 ELSE 0 END) AS attack_count
> FROM log_events
> WHERE label != 'normal'
> GROUP BY service
> ORDER BY attack_count DESC
> LIMIT 10;
```

```

MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 3.91 sec HDFS Read: 10223469 HDFS Write: 1725 SUCCESS
Stage-Stage-2: Map: 1 Reduce: 1 Cumulative CPU: 2.3 sec HDFS Read: 9795 HDFS Write: 372 SUCCESS
Total MapReduce CPU Time Spent: 6 seconds 210 msec
OK
ecr_i 281055 281055
private 103527 103527
http 2407 2407
other 1605 1605
eco_i 1253 1253
ftp_data 923 923
ftp 425 425
telnet 294 294
finger 202 202
csnet_ns 126 126
Time taken: 48.378 seconds, Fetched: 10 row(s)
hive>

```

9.6 — Analysis Query 4: Connection Flag Analysis

Script -

- SELECT flag, label, COUNT(*) AS count
- FROM log_events
- GROUP BY flag, label
- ORDER BY count DESC
- LIMIT 15;

Execution -

```

hive> SELECT flag, label, COUNT(*) AS count
> FROM log_events
> GROUP BY flag, label
> ORDER BY count DESC
> LIMIT 15;

```

```

MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 3.85 sec HDFS Read: 10220692 HDFS Write: 1889 SUCCESS
Stage-Stage-2: Map: 1 Reduce: 1 Cumulative CPU: 3.51 sec HDFS Read: 9927 HDFS Write: 508 SUCCESS
Total MapReduce CPU Time Spent: 7 seconds 360 msec
OK
SF smurf 280790
SF normal 91709
S0 neptune 86744
REJ neptune 20002
REJ normal 5341
SF back 2105
REJ satan 1229
SF ipsweep 1161
SF warezclient 1016
SF teardrop 979
RSTR portsweep 776
RSTO neptune 455
SF pod 264
REJ portsweep 220
SF satan 187
Time taken: 49.95 seconds, Fetched: 15 row(s)
hive>

```

Step 10 - Network Traffic Analysis Report

Summary of Findings:

Insight	Analysis
Dominant Attack Type	Smurf attacks make up 56.84% of the traffic (280,790 of ~494K connections), indicating a large-scale ICMP-based DDoS amplification attack.
Second Largest Threat	Neptune attacks account for 21.70% (107,201 connections), indicating a SYN flood targeting server resources.
Normal Traffic Baseline	Only 19.69% (97,278) of traffic is normal, showing the network is heavily compromised.
Protocol Risk	ICMP dominates with 283,602 connections, ~99.6% of which are malicious.
TCP vs UDP	TCP has 190,065 connections (113,252 attacks), while UDP is minimal with 20,354 connections and 1,177 attacks.
Most Abused Service	ecr_i service shows 281,055 connections — 100% malicious, linked to smurf attacks.
Stealthy Services	Services like private , http , eco_i , ftp_data , ftp , telnet , finger , csnet_ns show zero legitimate traffic.
Flag-Based Risk	SF flags dominate (smurf, normal), while REJ flags in neptune confirm SYN flood behavior.
Multi-Vector Attack Pattern	Patterns (SF+smurf, S0+neptune, REJ+neptune) indicate a coordinated multi-vector DDoS attack.

Critical Threats Identified:

Rank	Attack	Count	% of Traffic	Protocol
1	Smurf	280,790	56.84%	ICMP
2	Neptune	107,201	21.70%	TCP
3	Normal	97,278	19.69%	Mixed
4	Back	2,203	0.45%	TCP
5	Satan	1,589	0.32%	TCP
6	IPSweep	1,247	0.25%	ICMP/TCP
7	Portsweep	1,040	0.21%	TCP
8	Warezclient	1,020	0.21%	TCP
9	Teardrop	979	0.20%	UDP
10	Pod	264	0.05%	ICMP

MapReduce Performance Summary:

Stage	Map	Reduce	HDFS Read	HDFS Write	Status
Query 1 - Stage 1	1	1	10,220,235	716	SUCCESS
Query 1 - Stage 4	1	1	10,219,717	117	SUCCESS
Query 2 - Stage 1	1	1	10,222,591	182	SUCCESS
Query 3 - Stage 1	1	1	10,223,469	1,725	SUCCESS
Query 4 - Stage 1	1	1	10,220,692	1,889	SUCCESS

Total CPU Time across all queries: ~37 seconds processing ~10MB of network log data across all MapReduce stages.

Recommendations:

Priority	Recommendation
High	Block ICMP flood traffic at perimeter firewall — 99.6% of ICMP is malicious
High	Implement SYN cookies to counter Neptune (SYN flood) attacks on TCP
Medium	Monitor ecr_i, private, and telnet services — showing 100% attack traffic
Medium	Deploy rate limiting on all services showing zero legitimate connections
Low	Increase swap space and node memory for faster MapReduce processing in production